

GRAMATICA UTILIZADA

Descripción general de la Gramática:

La gramática fue diseñada para reconocer los elementos fundamentales definidos para la Fase 1 del lenguaje GoLite. El símbolo inicial corresponde a <INICIO>, desde donde se derivan todas las sentencias válidas del programa.

La estructura permite el reconocimiento de declaraciones de variables con y sin tipado explícito, asignaciones simples y compuestas, llamadas a la función embebida `fmt.Println`, estructuras condicionales `if-else`, ciclos `for`, sentencias `break` y `continue`, así como expresiones aritméticas, relacionales y lógicas.

Para evitar ambigüedades en la evaluación de expresiones, se definieron reglas de precedencia y asociatividad dentro del analizador sintáctico generado con CUP. Estas reglas garantizan que las operaciones sean interpretadas respetando el comportamiento esperado del lenguaje GoLite.

Adicionalmente, se incorporó un mecanismo básico de recuperación de errores mediante la producción especial `error`, permitiendo al analizador continuar procesando la entrada después de detectar una sentencia inválida y generar reportes de errores sintácticos más completos.

Descripción de la Gramática

A continuación se presentan las producciones principales junto con una breve descripción de su función dentro del análisis sintáctico.

Símbolo inicial

$::= \langle \text{LISTA_SENTENCIAS} \rangle$

Descripción:

Esta producción representa el punto de entrada del analizador sintáctico. Todo programa válido debe comenzar siendo reconocido como una lista de sentencias.

Lista de sentencias

$\langle \text{LISTA_SENTENCIAS} \rangle ::= \langle \text{LISTA_SENTENCIAS} \rangle$

|

Descripción:

Permite reconocer una o múltiples sentencias dentro del programa. Gracias a la recursividad por la izquierda es posible procesar secuencias de instrucciones de longitud indefinida.

Sentencias

$::=$

|

|

| $\langle \text{IF_SENTENCIA} \rangle$

| $\langle \text{FOR_SENTENCIA} \rangle$

| $\langle \text{BREAK_SENTENCIA} \rangle$

| $\langle \text{CONTINUE_SENTENCIA} \rangle$

|

Descripción:

Agrupar todas las estructuras válidas que pueden aparecer dentro de un programa. Cada sentencia representa una acción ejecutable del lenguaje.

Sentencia If

$\langle \text{IF_SENTENCIA} \rangle ::= \text{if "(" "}"$

| $\text{if "(" "}" \text{ else}$

Descripción:

Reconoce estructuras condicionales. La primera alternativa corresponde a un if simple y la segunda a un if-else. La condición debe estar formada por una expresión válida.

Ejemplos:

```
if(a > b){  
    fmt.Println(a);  
}
```

```
if(a > b){  
    fmt.Println(a);  
}else{  
    fmt.Println(b);  
}
```

Sentencia For

<FOR_SENTENCIA> ::= for "(" ")"

Descripción:

Permite reconocer ciclos for basados en una condición lógica. Mientras la expresión evaluada sea verdadera, el bloque asociado podrá ejecutarse repetidamente.

Ejemplo:

```
for(i < 10){  
    i += 1;  
}
```

Sentencia Break

<BREAK_SENTENCIA> ::= break ";"

Descripción:

Representa la instrucción utilizada para finalizar anticipadamente la ejecución de un ciclo.

Sentencia Continue

<CONTINUE_SENTENCIA> ::= continue ";"

Descripción:

Representa la instrucción utilizada para omitir el resto de instrucciones de la iteración actual y continuar con la siguiente iteración del ciclo.

Declaraciones

```
::= var identificador "=" ";"  
| var identificador "=" ";"  
| var identificador ";"
```

Descripción:

Permite declarar variables utilizando inferencia de tipo o especificando explícitamente el tipo de dato. También contempla declaraciones sin inicialización.

Ejemplos:

```
var edad = 20;  
var nombre string = "Juan";  
var activo bool;
```

Asignaciones

```
::= identificador "=" ";"  
| identificador "+=" ";"  
| identificador "-=" ";"
```

Descripción:

Reconoce las operaciones de modificación de variables. Incluye asignación simple y asignaciones compuestas mediante suma y resta.

Ejemplos:

```
x = 10;  
x += 5;  
x -= 3;
```

Impresión

```
::= fmt "." println "(" ")" ";"
```

Descripción:

Reconoce llamadas a la función embebida `fmt.Println` utilizada para mostrar información en la consola de salida.

Ejemplo:

```
fmt.Println("Hola Mundo");
```

Valores

```
::= entero  
| decimal  
| cadena  
| identificador  
| true  
| false  
| nil
```

| literal_rune
| <FUNCION_EMBEBIDA>

Descripción:

Representa los elementos básicos que pueden participar dentro de una expresión. Incluye literales, identificadores, valores booleanos, nil y llamadas a funciones embebidas.

Expresiones

::=
| "(" ")"
| "+"
| "-"
| "*"
| "/"
| "%"
| ">"
| "<"
| ">="
| "<="
| "=="
| "!="
| "&&"
| "||"
| "!"
| "-"

Descripción:

Es una de las producciones más importantes de la gramática. Permite construir expresiones aritméticas, relacionales y lógicas respetando las reglas de precedencia y asociatividad definidas en CUP.

Ejemplos:

5 + 3 * 2
a > b
x == y
true && false
!(a > b)

Bloques

::= "{ }"
| "{"
<LISTA_SENTENCIAS>
"}"

Descripción:

Permite agrupar múltiples instrucciones dentro de una misma estructura sintáctica. Los bloques son utilizados principalmente en sentencias de control como if y for.

Funciones embebidas

```
<FUNCION_EMBEBIDA> ::= strconv "." atoi "(" ")"  
| strconv "." parseFloat "(" ")"  
| reflect "." typeOf "(" ")"
```

Descripción:

Reconoce las funciones incorporadas en el lenguaje para conversión de tipos y obtención de información sobre tipos de datos.

Ejemplos:

```
strconv.Atoi("25")  
strconv.ParseFloat("3.14")  
reflect.TypeOf(variable)
```

Tipos de datos

```
::= int  
| float64  
| string  
| bool  
| rune
```

Descripción:

Define los tipos de datos primitivos soportados por la implementación del lenguaje GoLite desarrollada para este proyecto.